AgriDirect AI

Crop demand prediction · Direct farmer-to-company marketplace

TECHNICAL REQUIREMENTS DOCUMENT

Technical Requirements & Design

Architecture, data model, APIs, ML pipeline and engineering standards

Version	**1.0**
Date	**24 October 2026**
Companion to	**PRD v1.1, Security & Privacy v1.0, Research Report v1.0**
Audience	**Full-stack, ML and DevOps developers**
Status	**Draft for team review**

Confidential — prepared for the AgriDirect AI development team.

Contents

1. Purpose and principles . . . 3
2. System architecture . . . 3
3. Technology decisions . . . 4
4. Repository structure . . . 5
5. Data model . . . 5
6. API contract (v1) . . . 6
7. ML pipeline . . . 7
8. Frontend architecture . . . 8
9. Environments, CI/CD and DevOps . . . 9
10. Testing strategy . . . 9
11. Observability and operations . . . 9
12. Coding standards . . . 9
13. Effort estimate (indicative) . . . 10

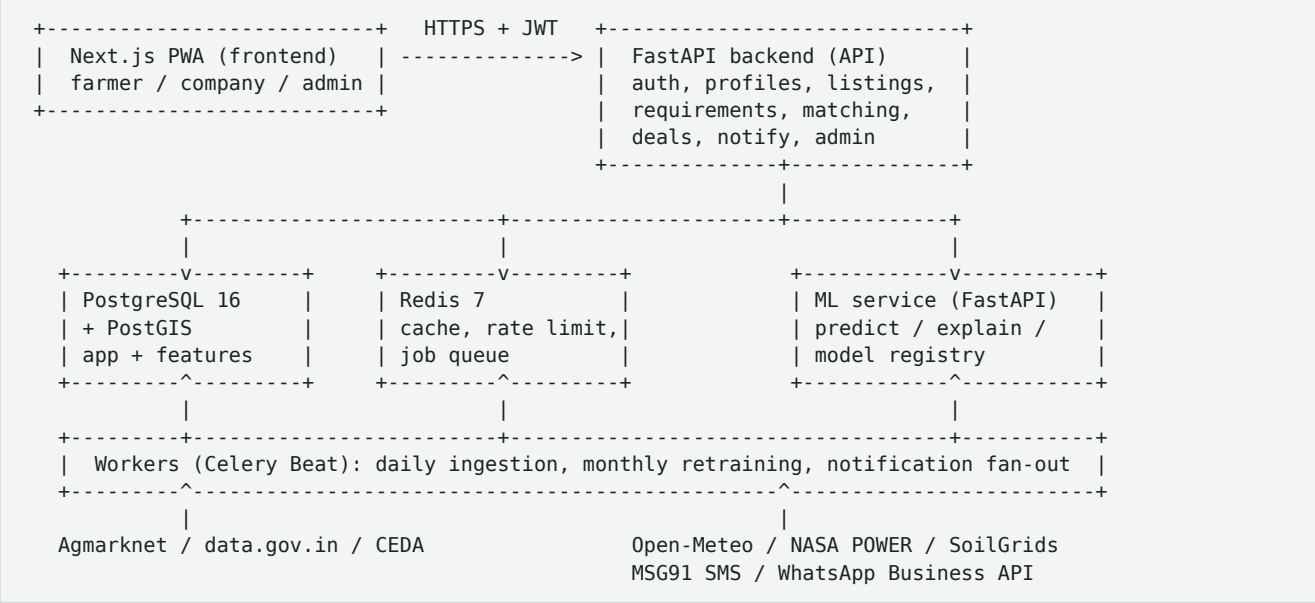
1. Purpose and principles

This document turns the PRD into a buildable design. It fixes the tech stack, defines the service boundaries, data model, API contract, ML pipeline and the standards every pull request must meet.

1.1 Design principles

- **Monolith first.** One FastAPI backend with clear internal modules; split into services only when load or team size demands it.
- **Python everywhere on the server.** API and ML share language, types and libraries.
- **Frontend works offline-tolerant.** PWA with cached shell; farmer screens usable on slow 3G/4G.
- **Advisory, never authoritative.** Every prediction carries confidence and model version.
- **Everything auditable.** Admin actions and deal status changes write to an append-only audit log.
- **Prototype → product.** The Phase 0 Next.js prototype is the real frontend; mock data is swapped for API calls behind a single data-access layer.

2. System architecture



2.1 Components

Component	Responsibility	Technology
Web / PWA client	Farmer, company and admin interfaces; offline shell; i18n	Next.js 14 (App Router), TypeScript, Tailwind CSS, next-intl, Recharts, Leaflet
API backend	Auth, RBAC, profiles, listings, requirements, matching, deals, notifications, admin	Python 3.12, FastAPI, SQLAlchemy 2, Pydantic v2, Alembic
ML service	Training, evaluation, prediction and explanation endpoints	FastAPI, LightGBM/XGBoost, statsmodels, Prophet, SHAP, MLflow
Workers	Scheduled ingestion, retraining, notification fan-out	Celery + Celery Beat, Redis broker
Database	Operational data, features, predictions, audit log	PostgreSQL 16 + PostGIS 3
Cache / queue	OTP rate limits, prediction cache, job queue	Redis 7
Object storage	Listing photos, documents, model artefacts	S3-compatible (AWS S3 ap-south-1 or equivalent)
Observability	Errors, logs, metrics, uptime	Sentry, structured JSON logs, Prometheus/Grafana or cloud equivalent

3. Technology decisions

Decision	Choice	Reason / alternative considered
Frontend framework	Next.js + TypeScript	SSR for fast first paint on 4G, file-based routing, PWA support. Alternative: Vite + React (simpler, no SSR).
Styling	Tailwind CSS	Small CSS bundle, consistent spacing; no runtime CSS-in-JS cost on low-end phones.
Backend	FastAPI	Async, auto OpenAPI docs, shares Python with ML. Alternative: NestJS (would split languages).
Database	PostgreSQL + PostGIS	Relational integrity for deals; ST_DWithin for distance queries.
Auth	Phone OTP via MSG91 + JWT	Indian DLT-compliant SMS; Firebase Auth acceptable alternative for prototype.
Maps	Leaflet + OpenStreetMap	No per-request cost; Google Maps optional later.
ML baseline	LightGBM	Strong on tabular lag features, fast to train, works with SHAP.
Hosting	AWS Mumbai (ap-south-1), Docker	Data stays in India; ECS/Fargate or a single EC2 + Docker Compose for MVP.
CI/CD	GitHub Actions	Lint, test, build images, deploy to staging on merge to main.

4. Repository structure

```
agridirect/
├── frontend/                # Next.js PWA (Phase 0 prototype lives here)
│   ├── app/                 # routes: /, /login, /farmer/*, /company/*, /admin/*
│   ├── components/          # UI kit: Button, Card, Badge, Stat, Chart, Map
│   ├── lib/
│   │   ├── api.ts           # single data-access layer (mock ↔ real switch)
│   │   └── mock/            # mock data used by the prototype
│   ├── messages/            # en.json, hi.json, mr.json
│   └── public/              # icons, manifest.webmanifest
├── backend/
│   ├── app/
│   │   ├── api/v1/          # routers: auth, farmers, companies, listings, ...
│   │   ├── core/            # config, security, rbac, rate_limit
│   │   ├── models/          # SQLAlchemy models
│   │   ├── schemas/         # Pydantic request/response models
│   │   ├── services/        # matching, deals, notifications
│   │   └── workers/         # celery tasks
│   ├── alembic/             # migrations
│   └── tests/
├── ml/
│   ├── pipelines/           # ingest_mandi.py, ingest_weather.py, ingest_soil.py
│   ├── features/            # feature engineering
│   ├── models/              # train.py, evaluate.py, model_card.md
│   └── serve/               # FastAPI prediction service
├── infra/                   # docker-compose.yml, Dockerfiles, terraform (later)
└── .github/workflows/       # ci.yml, deploy-staging.yml
```

> The frontend source delivered with this document already follows the **frontend/** layout. Swapping mock data for the real API means editing only **lib/api.ts**.

5. Data model

All tables have **id (UUID)**, **created_at**, **updated_at**. Quantities stored in kilograms (integer); prices in paise per quintal (integer) to avoid floating point errors.

Table	Key columns	Notes
users	role (farmer|company|admin), phone_e164 UNIQUE, name, language, status (pending|verified|blocked)	No password column; OTP only.
farmer_profiles	user_id FK, village, district_id FK, state, location GEOGRAPHY(Point), land_area_acres, irrigation, soil_type	Location index GIST.
company_profiles	user_id FK, company_name, type, gstin UNIQUE, operating_districts[], contact_person, verified_by, verified_at	gstin encrypted at rest.
crops	name_en, name_hi, name_mr, category, seasons[], duration_days, agmarknet_code	Admin-managed catalogue.
districts	name, state, centroid GEOGRAPHY(Point), agro_climatic_zone	Master data.
listings	farmer_id, crop_id, variety, qty_kg, qty_remaining_kg, ready_date, expected_price, location, status (active|paused|sold|expired)	Partial fulfilment via qty_remaining_kg.
listing_photos	listing_id, object_key, width, height	EXIF stripped on upload.
requirements	company_id, crop_id, qty_kg, required_by, delivery_location, price_min, price_max, quality_spec JSONB, status	

Table	Key columns	Notes
matches	listing_id, requirement_id, score NUMERIC(5,2), score_breakdown JSONB, status (suggested|dismissed|converted)	UNIQUE(listing_id, requirement_id).
deals	listing_id, requirement_id NULL, company_id, farmer_id, qty_kg, price, status	Row-level lock on status change.
deal_events	deal_id, actor_id, from_status, to_status, payload JSONB	Full negotiation history.
market_prices	crop_id, mandi_id, date, min_price, max_price, modal_price, arrivals_t	Partition by year; UNIQUE (crop,mandi,date).
weather_records / soil_records	location_key, date, params JSONB	Daily weather; soil static per grid cell.
predictions	crop_id, district_id, as_of, horizon_weeks, demand_level, trend, price_direction, confidence, model_version, explanation JSONB	Cached; recomputed daily.
notifications	user_id, channel, template, payload, status, sent_at	
reviews	deal_id, reviewer_id, rating 1–5, comment	Only after Completed.
audit_log	actor_id, action, entity, entity_id, before JSONB, after JSONB, ip	Append-only; no UPDATE/DELETE grants.

6. API contract (v1)

Base path **/api/v1**. JSON only. Auth via **Authorization: Bearer <access JWT>** (15-min expiry) with refresh token in an httpOnly, Secure, SameSite=Strict cookie (7-day). Errors follow RFC 7807 problem+json.

Method + path	Role	Purpose
POST /auth/otp/request	Public	Send OTP to phone (rate-limited)
POST /auth/otp/verify	Public	Verify OTP; returns access token, sets refresh cookie
POST /auth/refresh · POST /auth/logout	Auth	Rotate / revoke tokens
GET/PUT /farmers/me	Farmer	Farmer profile
GET/PUT /companies/me	Company	Company profile + verification documents
POST /advice/crops	Farmer	Ranked crop recommendations
GET /predictions/demand?crop=&district=	Auth	Demand level, trend, price direction, confidence
GET /market/prices?crop=&mandi=&from=&to=	Auth	Price and arrival series
POST/GET /listings · GET/PATCH /listings/{id}	Farmer (write), Company (read)	Produce listings; search with filters
POST/GET /requirements · PATCH /requirements/{id}	Verified company	Requirements
GET /matches?listing_id= | requirement_id=	Owner	Suggested matches
POST /deals · PATCH /deals/{id}/status · GET /deals/{id}	Parties	Create offer, transition status, view history
POST /deals/{id}/review	Parties	Rating after Completed

Method + path	Role	Purpose
GET /admin/users · PATCH /admin/users/{id}/verify	Admin	Verification and moderation
GET /admin/system	Admin	Pipeline and model health

6.1 Example — crop advice

```
POST /api/v1/advice/crops
{ "district_id": "d-pune", "land_area_acres": 4, "season": "rabi", "irrigation": "well" }

200 OK
{ "as_of": "2026-10-24", "model_version": "lgbm-2026.10.1",
  "recommendations": [
    { "crop": "maize", "rank": 1, "suitability": 0.86,
      "demand_level": "high", "trend": "rising", "price_direction": "up",
      "confidence": 0.74,
      "explanation": "Demand expected to rise: local arrivals down 18% vs last month and 3 buyers need 140 t within 6
    ...
  ],
  "disclaimer": "Advisory only. Prices and demand can change." }
```

6.2 Example — change deal status

```
PATCH /api/v1/deals/7f3c.../status
{ "to": "negotiation", "price_per_quintal": 2250, "note": "Can deliver by 12 Nov" }

409 Conflict  -> transition not allowed from current status
403 Forbidden -> caller is not the party allowed to act
```

7. ML pipeline

7.1 Data ingestion (daily, 02:00 IST)

Source	Method	Stored as
Agmarknet via data.gov.in OGD API (key required)	Daily pull per state/commodity; retry with backoff; fallback to CEDA Agri Market API (historical) and admin CSV upload	market_prices
Open-Meteo forecast + archive (ERA5)	Per district centroid; 16-day forecast + daily history	weather_records
NASA POWER (backup)	Daily agro-climate parameters	weather_records
ISRIC SoilGrids 250 m / Soil Health Card summaries	One-time per district grid; refresh yearly	soil_records
Platform requirements	Live from DB	Aggregated into features

7.2 Features (per crop × market × week)

- Price lags (1, 2, 4, 8, 52 weeks), rolling mean/std, % change.
- Arrival lags and arrival anomaly vs 3-year same-week average (supply glut signal).
- Weather: cumulative rainfall vs normal, max temperature, forecast rainfall next 2 weeks.
- Calendar: week of year, season, festival flags (Diwali, Dussehra, etc.).
- Soil: pH, organic carbon, texture class (for suitability model).

- Platform demand: open requirement tonnage within 50/100 km.

7.3 Models

Model	Target	Approach
Price forecaster	Modal price at t+2…t+12 weeks	Seasonal-naive baseline → LightGBM direct multi-horizon; compare SARIMAX/Prophet; pick per crop by validation.
Demand classifier	Low/Medium/High demand; Falling/Stable/Rising trend	Derived from forecast price change and arrival anomaly thresholds; LightGBM classifier when labels mature.
Crop suitability	0–1 suitability score	Rule base from ICAR crop calendars (season, rainfall, soil) — later learned.
Recommender	Ranked crops	score = 0.5 × suitability + 0.35 × demand/price score + 0.15 × platform demand; ties broken by confidence.
Explanation	Text in EN/HI/MR	Top-3 SHAP features → sentence templates per language.

7.4 Validation

- Rolling-origin (walk-forward) time splits; never random splits.
- Report MAPE, RMSE (price) and macro-F1 (class) per crop, district and season, vs seasonal-naive baseline.
- Confidence = calibrated from historical error of the same crop × horizon (e.g., 1 − normalised MAPE), shown as Low / Medium / High.
- Model card committed with each release: data window, metrics, known limitations.

7.5 Matching score

```
score = 100 × ( 0.30·crop_match           # 1 if same crop (0.6 if same crop family)
              + 0.25·qty_fit              # min(listing_qty, req_qty) / req_qty
              + 0.20·distance_score       # max(0, 1 − km / max_radius_km)
              + 0.15·date_fit             # 1 if ready ≤ required_by, decays 0.1/day late
              + 0.10·quality_fit )        # share of quality specs satisfied
Only pairs with crop_match > 0 and distance ≤ 200 km are scored. Recomputed on create/update.
```

8. Frontend architecture

- **Routing:** App Router with route groups (farmer), (company), (admin); layout per role with bottom navigation on mobile.
- **Data layer:** *lib/api.ts* exposes typed functions (getAdvice, listListings, createDeal…). Prototype returns mock data; production calls fetch with the JWT.
- **State:** React state + context for session; TanStack Query added when wired to API.
- **i18n:** next-intl with en/hi/mr message files; no hard-coded strings in components.
- **PWA:** manifest, service worker caching app shell and last-viewed advice.
- **Accessibility:** WCAG 2.1 AA contrast, 48 px touch targets, labels on all inputs, icons always paired with text.
- **Performance budget:** < 170 KB JS gzipped on farmer routes; images WebP, lazy-loaded.

9. Environments, CI/CD and DevOps

Environment	Purpose	Data
Local	Docker Compose: api, ml, postgres, redis, frontend	Seed script with fake users
Staging	Auto-deploy on merge to main; weekly demo	Anonymised / synthetic
Production	Manual approval deploy from tagged release	Real; backups daily, 30-day retention

9.1 Pipeline

1. Lint and type-check (ruff, mypy, eslint, tsc).
2. Unit tests (pytest, vitest) — fail under 70% coverage on core modules.
3. Dependency and secret scanning (pip-audit, npm audit, gitleaks).
4. Build Docker images, tag with git SHA, push to registry.
5. Run Alembic migrations, deploy to staging, run smoke tests.
6. Production deploy on release tag with manual approval.

10. Testing strategy

Level	Scope	Tool
Unit	Matching score, deal state machine, validators, feature engineering	pytest, vitest
API / integration	Each endpoint incl. RBAC and error cases against test DB	pytest + httpx, testcontainers
ML	Data quality checks, no leakage, metric regression vs last model	great_expectations / pandera, pytest
End-to-end	PRD demo script (register → deal accepted)	Playwright
Performance	1,000 concurrent users; prediction p95 < 2 s	k6 / Locust
Security	OWASP ZAP baseline, auth tests, dependency scan	ZAP, pip-audit, npm audit
Devices	3 Android (one 2 GB RAM), Chrome, Safari	Manual + BrowserStack

11. Observability and operations

- Structured JSON logs with request ID; no phone numbers or OTPs in logs.
- Sentry for backend and frontend errors; alert on new issue in production.
- Metrics: request latency p50/p95, error rate, OTP send success, pipeline duration, model MAPE drift.
- Alerts: pipeline failure, API error rate > 2% for 5 min, prediction latency p95 > 2 s, disk > 80%.
- Runbook in repo for: pipeline failure, SMS provider outage, DB restore.

12. Coding standards

- Conventional commits; PRs need one review and green CI.
- Python: ruff + black formatting, type hints, no business logic in routers.
- TypeScript: strict mode, ESLint, components < 200 lines, no *any*.
- Config via environment variables only; .env never committed; secrets in cloud secret manager.

- Every new endpoint ships with OpenAPI description, tests and RBAC check.

13. Effort estimate (indicative)

Work package	Person-days
Phase 0 prototype (done as part of this delivery) + polish	3–5
Auth, RBAC, profiles, verification	8
Data pipelines (mandi, weather, soil) + feasibility	8
Models, validation report, prediction/advice API	12
Listings, requirements, matching, deals, notifications	12
Frontend wiring, i18n (HI/MR), PWA	8
Admin dashboard and system health	5
Testing, security hardening, UAT fixes	8
Deployment, docs, handover	4
Total	**68–70**